Machine Learning — Lecture 7

Neural Networks and Backpropagation

FCIS — For Course Use Only

Dr. Fayçal Hamdi

Faculty of Computer and Information Systems

Lecture 7 at a Glance

FCIS — For Classroom Use Only

Neural Networks and Backpropagation

Course Learning Outcomes

CLO3: Construct and analyze machine learning models by applying algorithmic principles and internal computations.

CLO4: Apply supervised learning techniques to solve practical problems and interpret their outcomes.

Lecture purpose

- Connect neural networks to models already studied.
- Understand neurons, layers, activations, and forward propagation.
- Explain how loss flows backward through the network.
- Perform a small backpropagation calculation.
- Understand training choices, overfitting, and practical limitations.

FCIS — For Course Use Only

💡 Main idea: a **neural network** is a composition of simple differentiable units. Backpropagation efficiently computes how each weight contributed to the loss.

Learning Objectives

FCIS — Course Use Only

By the end of Lecture 7, you will be able to:

1. Explain a neuron as a weighted sum followed by an activation.
2. Distinguish input, hidden, and output layers.
3. Compute a forward pass through a small neural network.
4. Choose output activations for regression, binary classification, and multiclass classification.
5. Explain why **activation functions** are necessary.
6. Describe backpropagation using the chain rule.
7. Compute output-layer and hidden-layer gradients for a small network.
8. Update weights and biases using gradient descent.
9. Explain learning rate, epochs, batches, initialization, and training curves.
10. Discuss overfitting, regularization, and practical limitations of neural networks.

FCIS — Course Use Only

Lecture Roadmap

FCIS — Course Use Only

From linear models to multilayer learning

Learning phase	Parts	Core question
1. Understand neural networks	**I–III**	Why do we need hidden layers, and how do neurons, activations, and network architectures work?
2. Predict and measure error	**IV–V**	How does a forward pass produce a prediction, and which loss measures its error?
3. Learn by backpropagation	**VI–VII**	How does the chain rule assign gradients to every weight and bias?
4. Train, generalize, and apply	**VIII–XI**	How do optimization, regularization, preprocessing, and model choice affect reliable performance?

FCIS — For Course Use Only

💡 **Practice & consolidation:** Part XII connects neuron calculations, gradients, output choices, training diagnostics, and common mistakes.

Part I

FCIS — For Students Use Only

Why Neural Networks?

Neural networks combine many simple computations to learn complex nonlinear patterns.

FCIS — For Course Use Only

From Previous Models to Neural Networks

For Course Use Only

What we already know

Both linear and logistic regression begin with a **weighted sum**:

- **Linear regression:** $\hat{y} = w^\top x + b$
- **Logistic regression:** $p = \sigma(w^\top x + b)$

A **neural network** connects many such units, repeatedly applying weighted sums and activation functions.

Weighted Sum
Combine inputs using weights and bias

→

Activation
Apply a nonlinear function

→

New Representation
Pass learned features forward

→

Prediction
Produce the final output

FCIS — For Course Use Only

💡 **Takeaway.** Neural networks extend the weighted-sum idea by combining many units across layers with **nonlinear activations**.

Why Add Hidden Layers?

FCIS — For Educational Use Only

Learning intermediate representations

A **hidden layer** transforms input features into new **learned representations** that help the network make predictions.

Example: Image classification

Different layers may learn increasingly complex patterns:

- **Early layers:** Detect simple patterns, such as edges.
- **Intermediate layers:** Combine patterns into shapes or textures.
- **Later layers:** Capture more complex, object-related features.

For **tabular data**, hidden layers can learn nonlinear combinations and interactions among input features.

> 💡 **Key idea.** Neural networks learn not only the final prediction, but also the **intermediate representations** used to produce it.

Linear Models Have Limits

FCIS — Private Use Only

When a straight boundary is not enough

Linear and logistic regression are strong baselines, but they are limited to linear relationships in their supplied features.

Neural networks use **hidden layers with nonlinear activations** to learn more complex relationships.

Examples

Data pattern	Limitation of a linear model
Curved relationships	Requires nonlinear feature transformations
Feature interactions	Often requires explicit interaction terms
Image patterns	Raw pixels may need spatial feature extraction
Text or speech	Complex representations may need to be learned

FCIS — For Course Use Only

💡 **Key idea.** Neural networks can **learn nonlinear features automatically**, reducing the need for manual feature engineering—but good data and careful evaluation remain essential.

Plot: Why Hidden Units Matter

FCIS — Academic Use Only

Two ReLU units can represent a pattern, a straight line cannot



FCIS — For Course Use Only

Constructed example, not a trained network: $h_1 = ReLU(x_1 - x_2)$ and $h_2 = ReLU(x_2 - x_1)$. A class-1 decision when $h_1 + h_2 \geq 0.5$ separates the four XOR corners. No *single straight-line boundary* can do so.

Neural Networks Are Still Models

FCIS — Course Use Only

They do not escape ML principles

Neural networks are powerful, but they must follow the same principles as other machine learning models.

They still require

- Appropriate **training data**;
- Separate **validation data**;
- Suitable preprocessing;
- A well-defined **loss function**;
- Effective optimization;
- Regularization when needed;
- Relevant evaluation metrics;
- Careful interpretation of results.

They can still suffer from

- **Underfitting:** Failing to learn patterns;
- **Overfitting:** Memorizing training details;
- **Data leakage:** Using unavailable information;
- **Distribution shift:** Struggling on changed data;
- **Bias or unreliability:** Producing problematic predictions.

FCIS — For Course Use Only

Key lesson. A neural network is **not automatically better** than a simpler model. Choose it when **validation performance, generalization, and application requirements** justify its added complexity.

Part II

FCIS — For Private Use Only

The Artificial Neuron

The basic unit of a **neural network** is a neuron: a weighted sum followed by a nonlinear activation.

FCIS — For Course Use Only

Neuron Computation

FCIS — Course Use Only

Weighted sum and activation

An artificial neuron transforms its input features through **two steps**:

$$\text{1. Weighted sum:} \quad z = w^\top x + b = \sum_{j=1}^{p} w_j x_j + b$$

$$\text{2. Activation:} \quad a = \phi(z)$$

What the symbols mean

Inputs and parameters

- x: Vector of input features.
- w: Weights controlling input contributions.
- b: Bias term.

Computation and output

- z: Weighted sum plus bias (**pre-activation**).
- ϕ: Activation function.
- a: Neuron output after activation.

FCIS — For Course Use Only

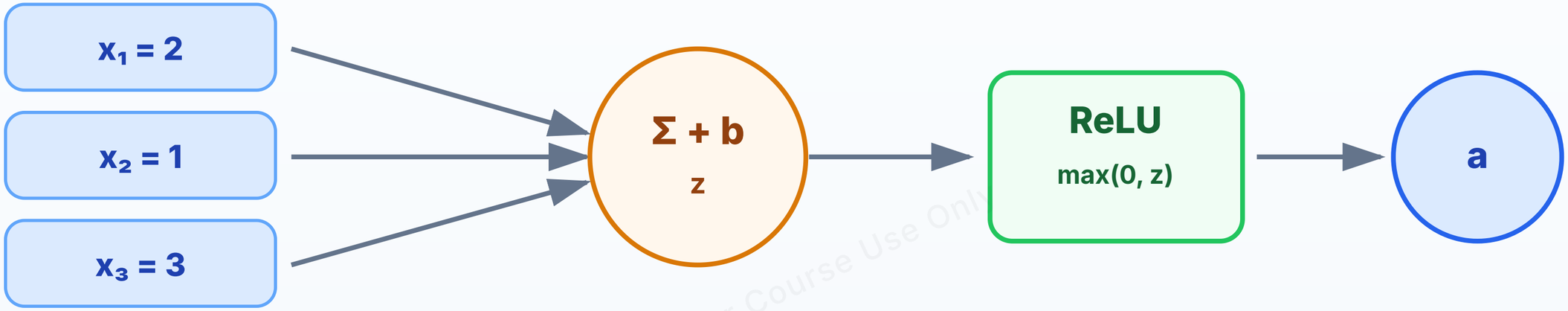
💡 **Key idea.** The weighted sum **combines the inputs**, while the activation function **transforms the result**. Nonlinear activations allow neural networks to learn more complex relationships.

Visual Walkthrough: One Artificial Neuron

For Course Use Only

Weighted inputs become an activated output

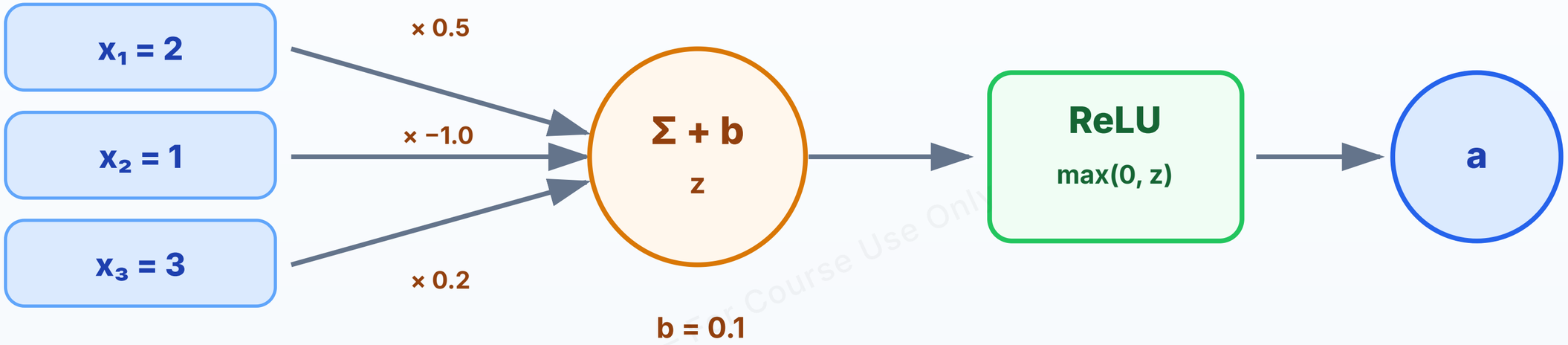
A neuron: weighted sum, bias, then activation



FCIS — For Course Use Only

Weighted inputs become an activated output

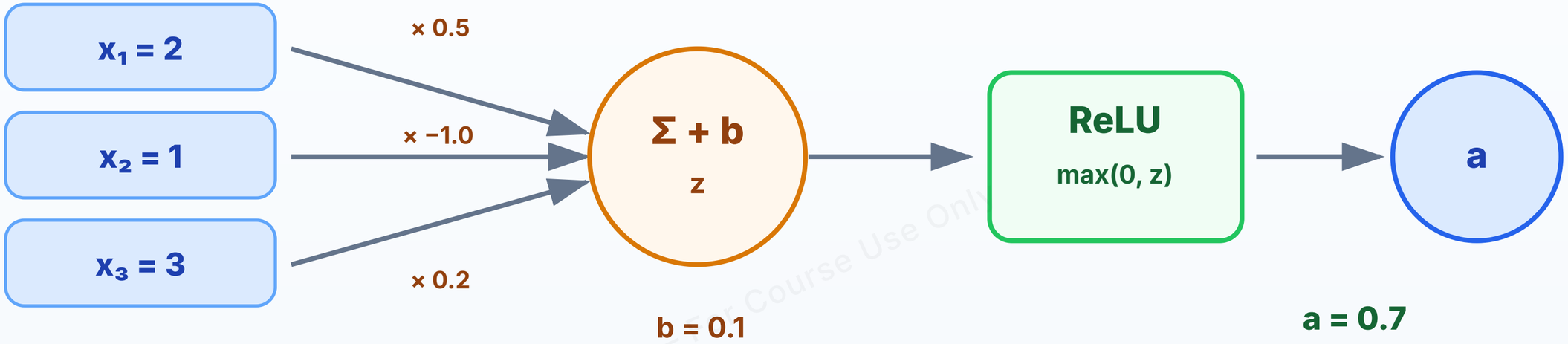
A neuron: weighted sum, bias, then activation



FCIS — For Course Use Only

Weighted inputs become an activated output

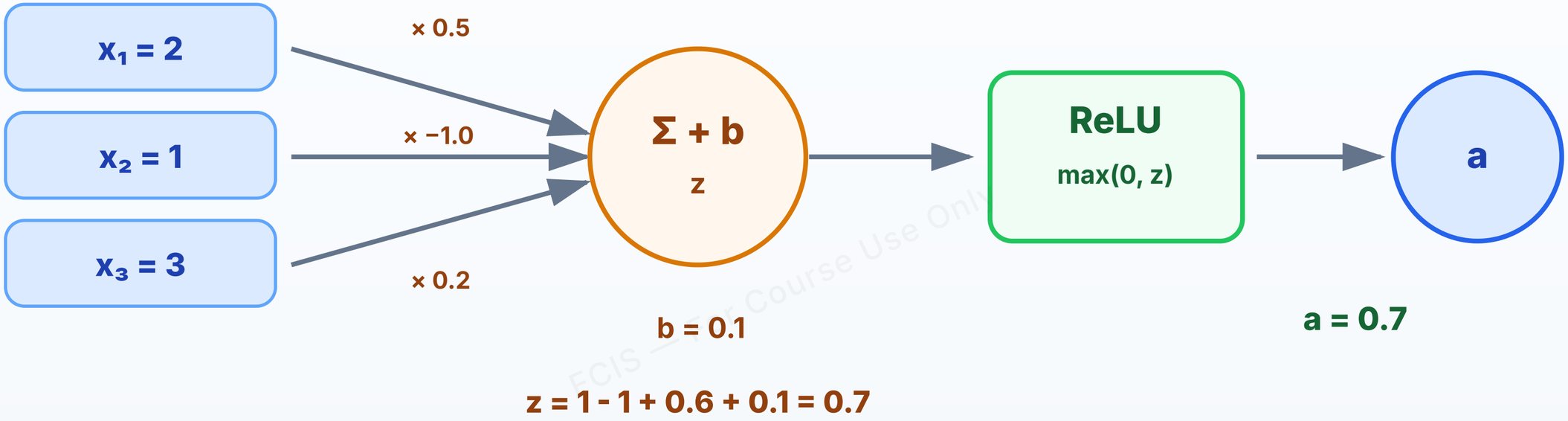
A neuron: weighted sum, bias, then activation



ECIS — For Course Use Only

Weighted inputs become an activated output

A neuron: weighted sum, bias, then activation



ECIS — For Course Use Only

💡 **Takeaway.** Each input contributes according to its **weight**, and the **bias** adjusts the sum. Here, $z = 0.7$ is positive, so ReLU preserves it: $a = \max(0, 0.7) = 0.7$.

Why Bias Matters

FCIS — Students Use Only

Shifting the activation

A neuron computes a weighted input and then applies an activation function:

$$z = wx + b, \qquad a = \phi(z)$$

- **Weight w:** Controls how strongly the input influences the neuron.
- **Bias b:** Shifts the neuron's response without changing the weight.

Example: One input, fixed weight $w = 1$

Bias	Weighted input	Where $z = 0$
$b = 0$	$z = x$	$x = 0$
$b = -2$	$z = x - 2$	$x = 2$

FCIS — For Course Use Only

Changing the bias shifts where the activation responds. For example, it shifts the **starting point of ReLU** or the **midpoint of sigmoid**.

> 💡 **Key idea.** The bias acts like the **intercept in linear regression**. It allows the neuron to adjust its response independently of the input weights.

Worked Neuron Example

FCIS — Course Use Only

Compute one neuron output

Suppose a neuron receives three inputs:

$$x = \begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix}, \qquad w = \begin{bmatrix} 0.5 \\ -1.0 \\ 0.2 \end{bmatrix}, \qquad b = 0.1$$

Step 1: Compute the weighted sum

$$\begin{aligned} z &= w^\top x + b \\ &= 0.5(2) - 1.0(1) + 0.2(3) + 0.1 \\ &= 1 - 1 + 0.6 + 0.1 = \boxed{0.7} \end{aligned}$$

FCIS — For Classroom Use Only

Step 2: Apply ReLU

$$a = \mathrm{ReLU}(z) = \max(0, 0.7) = \boxed{0.7}$$

💡 **Key idea.** A neuron first computes a **weighted sum plus bias**, then applies an **activation function** to produce its output.

Activation Functions

FCIS — Internal Use Only

Why do hidden layers need nonlinearity?

Activation functions allow neural networks to learn relationships beyond a single linear model.

Suppose two layers use **no nonlinear activation**:

$$h = W_1 x + b_1$$
$$\hat{y} = W_2 h + b_2$$

Substituting the first equation into the second:

$$\hat{y} = \underbrace{(W_2 W_1)}_{\text{new weights}} x + \underbrace{(W_2 b_1 + b_2)}_{\text{new bias}}$$

Faculty Course Use Only

This is still **one affine transformation** of the input.

> 💡 **Key idea.** Without nonlinear activations, stacking affine layers does not add expressive power beyond a single affine layer. **Nonlinearity enables more complex relationships.**

Sigmoid Activation

FCIS — For Course Use Only

Smooth output between 0 and 1

The **sigmoid function** converts any real-valued score into a value between 0 and 1:

$$\boxed{\sigma(z) = \frac{1}{1 + e^{-z}}}$$

Key properties

- **Range:** $(0, 1)$, with $\sigma(0) = 0.5$.
- **Smooth and increasing:** Larger inputs produce larger outputs.
- **Typical use:** Binary-classification output layers.

FCIS — For Course Use Only

Its derivative has a simple form:

$$\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big)$$

Limitation: Saturation. For very positive or negative inputs, the derivative approaches **zero**, which can slow learning in deep hidden layers.

Tanh Activation

FCIS — For Class Use Only

A zero-centered alternative

The **hyperbolic tangent** is another smooth nonlinear activation:

$$\boxed{\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}}$$

Key properties

- **Range:** $(-1, 1)$, with $\tanh(0) = 0$.
- **Zero-centered:** Outputs can be negative or positive.
- **Typical use:** Hidden layers, including some recurrent networks.

FCIS — For Course Use Only

Compared with sigmoid

Unlike sigmoid, tanh produces **negative outputs for negative inputs**. However, both functions saturate for large input magnitudes.

> 💡 **Key idea.** Tanh is **nonlinear and zero-centered**, but its gradients can still vanish when the activation saturates.

ReLU Activation

FCIS — For Educational Use Only

Simple and widely used

The **Rectified Linear Unit (ReLU)** is defined as:

$$\boxed{\mathrm{ReLU}(z) = \max(0, z)}$$

How does it behave?

Input	Output
$z \leq 0$	0
$z > 0$	z

FCIS — For Course Use Only

Advantages

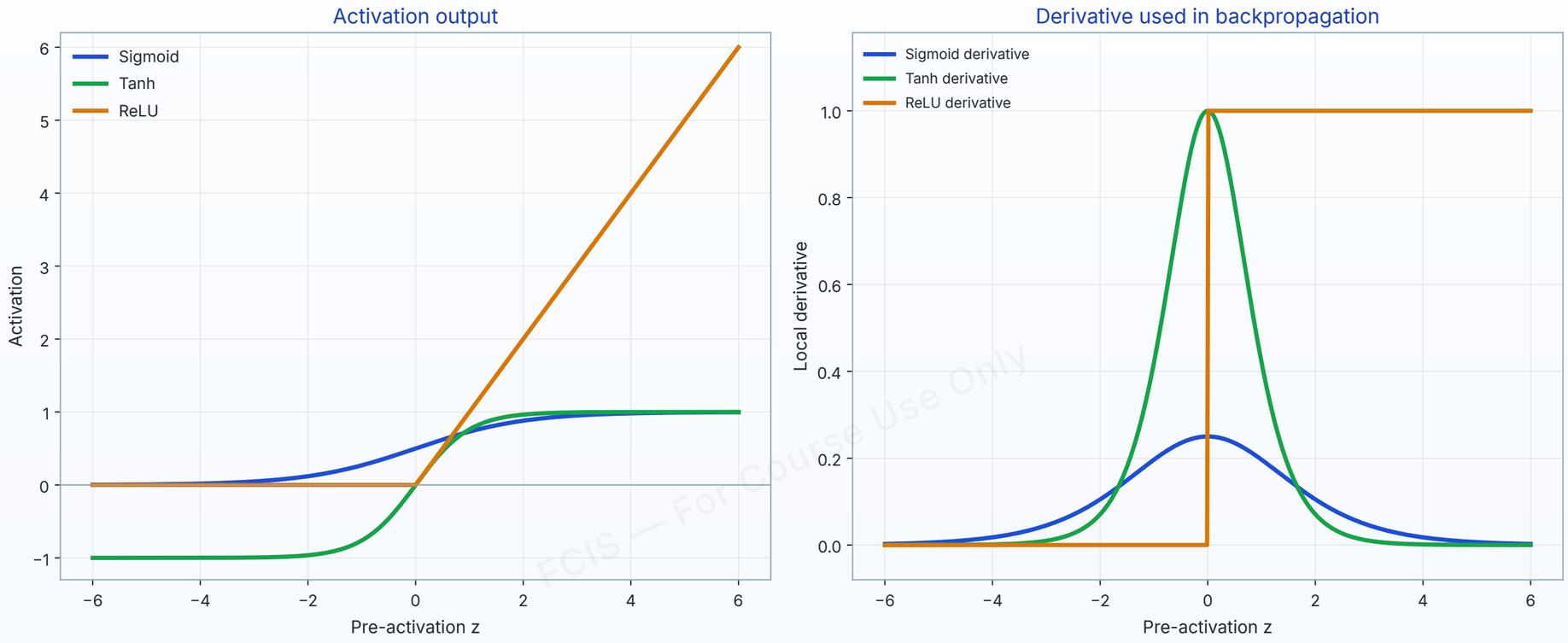
- **Efficient:** Requires only a simple comparison.
- **Effective gradients:** Derivative is 1 for positive inputs.
- **Widely used:** A common default for hidden layers.

Limitation: Inactive neurons. ReLU has zero gradient for negative inputs. Some neurons can become persistently inactive during training.

Plot: Activation Functions and Gradients

For Course Use Only

Shape controls both the signal and gradient flow



FCIS — For Course Use Only

💡 **Key idea. Sigmoid and tanh** saturate for large $|z|$, causing small derivatives and potentially weak gradients. **ReLU** has slope 1 for $z > 0$ and 0 for $z < 0$, allowing gradients through positive activations but blocking them through negative ones. At $z = 0$, a derivative convention is used.

Choosing Activations

FCIS — Class Use Only

Hidden vs. output layers

Hidden layers learn useful representations, while the **output layer** produces predictions appropriate to the task.

Layer role	Common activation	Why use it?
Hidden layer	**ReLU**	Introduces nonlinearity and supports efficient learning
Binary classification	**Sigmoid**	Outputs a value between 0 and 1, interpreted as the positive-class probability
Multiclass classification	**Softmax**	Produces probabilities across mutually exclusive classes that sum to 1
Regression	**Linear (identity)**	Allows unrestricted real-valued predictions

FCIS — For Course Use Only

💡 **Key idea.** Hidden-layer activations help the network **learn nonlinear representations**. The output activation should match the **prediction task and loss function**.

Part III

FCIS — For Private Use Only

Network Architecture

A **neural network** is built by organizing neurons into layers.

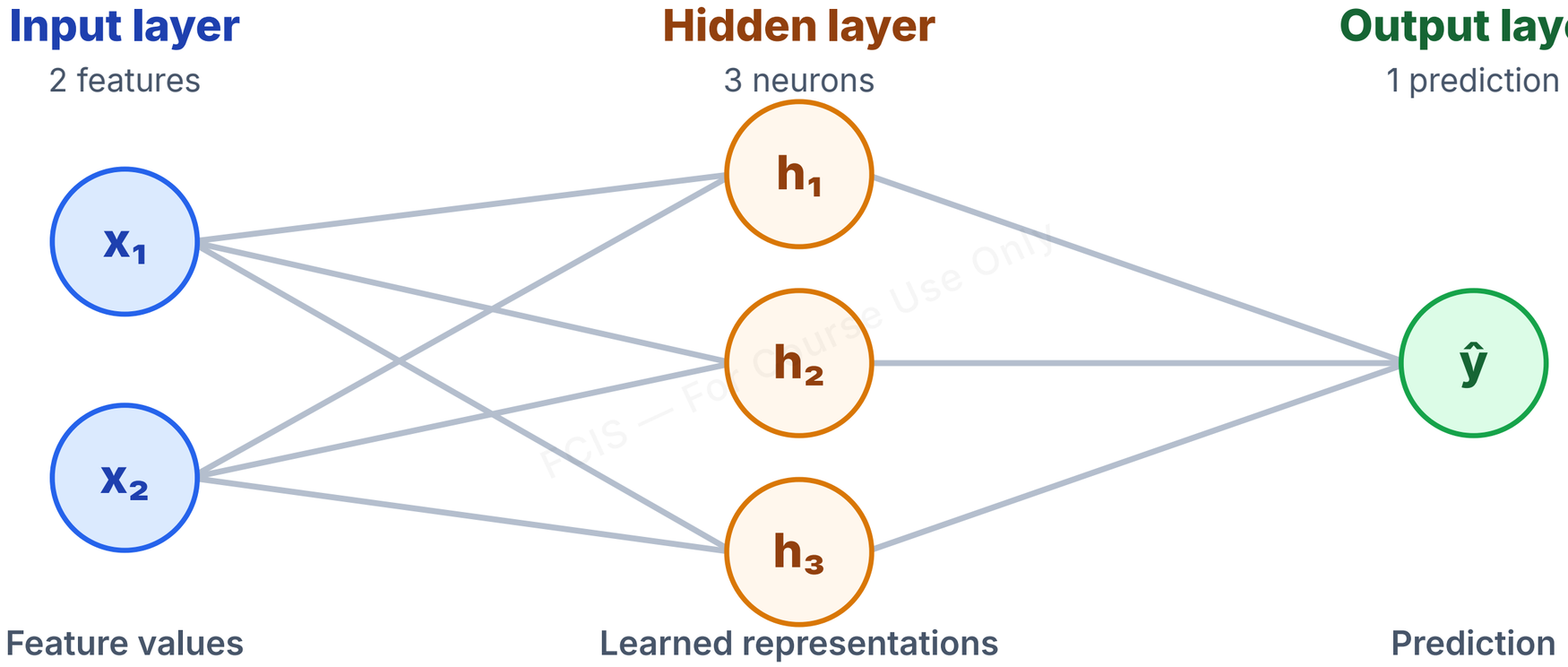
FCIS — For Course Use Only

Layers

FCIS — Course Use Only

Input, hidden, output

A **feedforward neural network** passes information from the input layer through hidden layers to the output layer.



ACIS — For Course Use Only

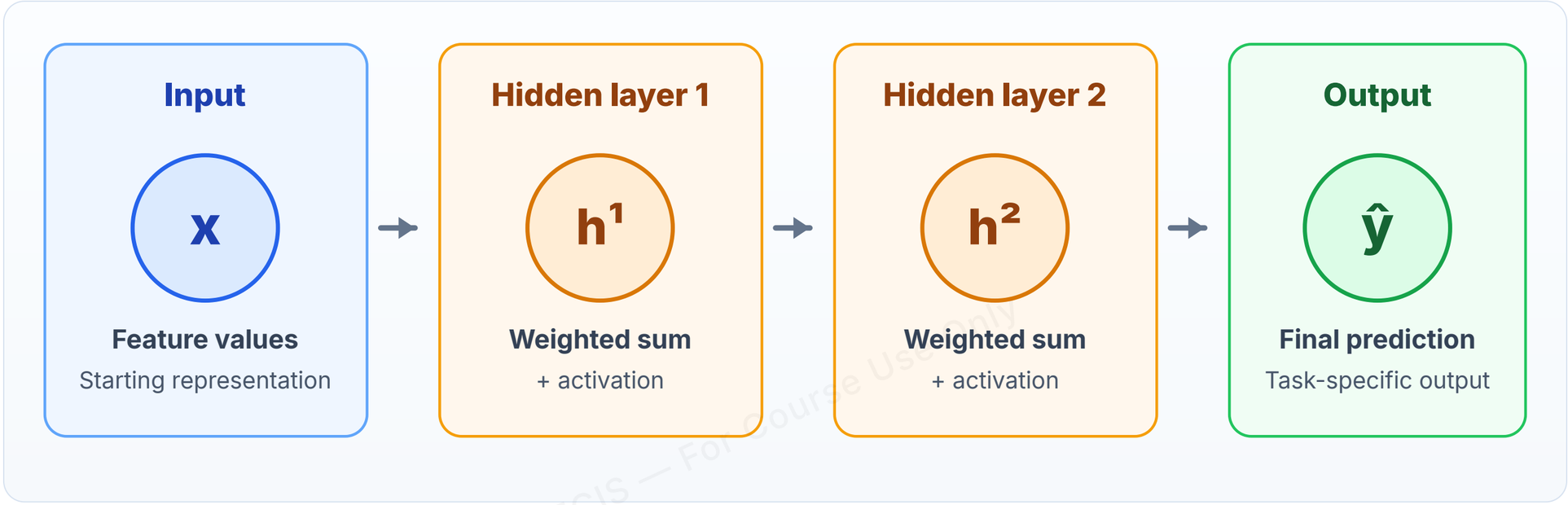
💡 **Key idea.** Hidden neurons combine inputs to learn intermediate representations. **Deep learning** uses multiple hidden layers to learn increasingly complex representations.

Feedforward Network

FCIS — For Course Use Only

Information flows forward

A **feedforward neural network** computes predictions **layer by layer**, without cycles or feedback connections.



FCIS — For Course Use Only

At each hidden layer, the network applies a learned transformation:

$$h^{(\ell)} = \phi\left(W^{(\ell)} h^{(\ell-1)} + b^{(\ell)}\right), \qquad h^{(0)} = x$$

Key distinction. Forward propagation computes predictions. **Backpropagation** computes gradients, which the optimizer uses to update the network's parameters.

Matrix Form for One Layer

FCIS — Course Use Only

Representing many neurons at once

A neural-network layer computes **all neuron outputs together**:

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}$$
$$a^{(\ell)} = \phi\left(z^{(\ell)}\right)$$

Understanding the notation

- **Input:** $a^{(\ell-1)}$ contains activations from the previous layer; $a^{(0)} = x$.
- **Parameters:** $W^{(\ell)}$ is the weight matrix and $b^{(\ell)}$ is the bias vector.
- **Output:** $z^{(\ell)}$ contains linear scores; $a^{(\ell)}$ contains activated outputs.

For Course Use Only

💡 **Key idea.** Matrix multiplication computes all neuron scores at once, then the activation function is applied **elementwise**.

Parameter Count

FCIS — Private Use Only

How many values does a layer learn?

Suppose a fully connected layer has m **inputs** and r **neurons**.

Parameter	Shape	Number of values
Weights W	$r \times m$	rm
Biases b	$r \times 1$	r
Total		$r(m+1)$

Worked example

A layer with **5 inputs and 3 neurons** has:

$$\underbrace{3 \times 5}_{\text{weights}} + \underbrace{3}_{\text{biases}} = \boxed{18 \text{ parameters}}$$

ECIS — For Course Use Only

💡 **Key idea.** Each neuron learns **one weight per input plus one bias**. The parameter count grows with both the number of inputs and neurons.

Output Layer by Task

FCIS — Private Use Only

Match predictions and loss to the task

The **output layer** determines the type of prediction a neural network produces.

Task	Typical output	Activation	Common loss
Regression	One continuous value	Linear	MSE / MAE
Binary classification	One probability	Sigmoid	Binary cross-entropy
Multiclass classification	K class probabilities	Softmax	Categorical cross-entropy

How are classification probabilities produced?

- **Sigmoid:** $p = \sigma(z)$, giving the probability of class 1.
- **Softmax:** $p_k = \dfrac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$, giving class probabilities that sum to 1.

ECIS — Course Use Only

💡 **Key idea.** Choose the output activation and loss according to **what the model must predict**, not simply the architecture of its hidden layers.

Part IV

FCIS — Personal Use Only

Forward Propagation

Forward propagation computes the network prediction from input features.

FCIS — For Course Use Only

Forward Pass Steps

FCIS — Non-Commercial Use Only

One hidden layer

A network with **one hidden layer** computes its prediction in two stages:

1. Hidden layer

Compute the weighted sum:

$$z^{(1)} = W^{(1)}x + b^{(1)}$$

Apply the hidden activation:

$$a^{(1)} = \phi(z^{(1)})$$

2. Output layer

Combine the hidden activations:

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}$$

Produce the prediction:

$$\hat{y} = g(z^{(2)})$$

FCIS — For Course Use Only

Putting it together

The complete network is a composition of transformations:

$$\boxed{\hat{y} = g\left(W^{(2)}\phi(W^{(1)}x + b^{(1)}) + b^{(2)}\right)}$$

💡 **Key idea.** Each layer performs a **weighted sum followed by an activation**. The hidden activation ϕ learns nonlinear representations, while the output activation g is chosen to match the prediction task.

Visual Walkthrough: Forward Pass

FCIS — Educational Use Only

The same worked network, revealed layer by layer

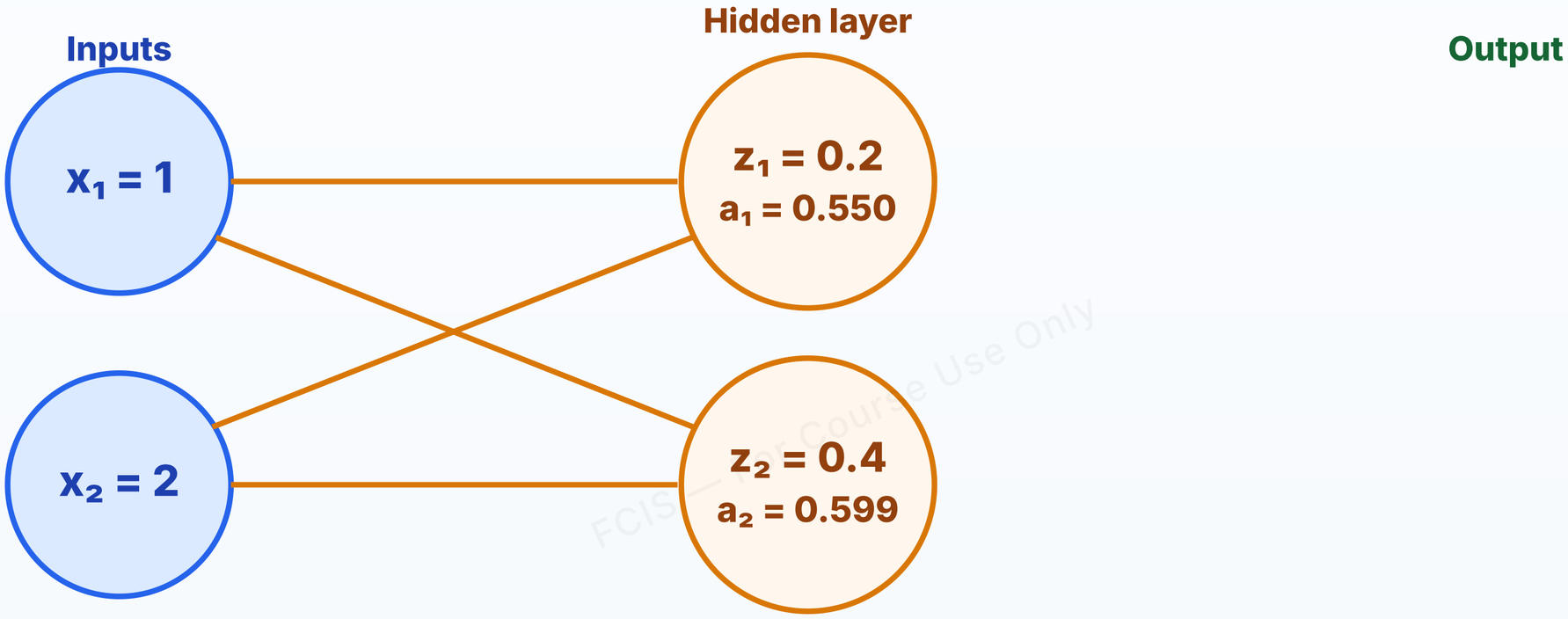
Input → hidden sigmoid units → output probability



FCIS — For Course Use Only

The same worked network, revealed layer by layer

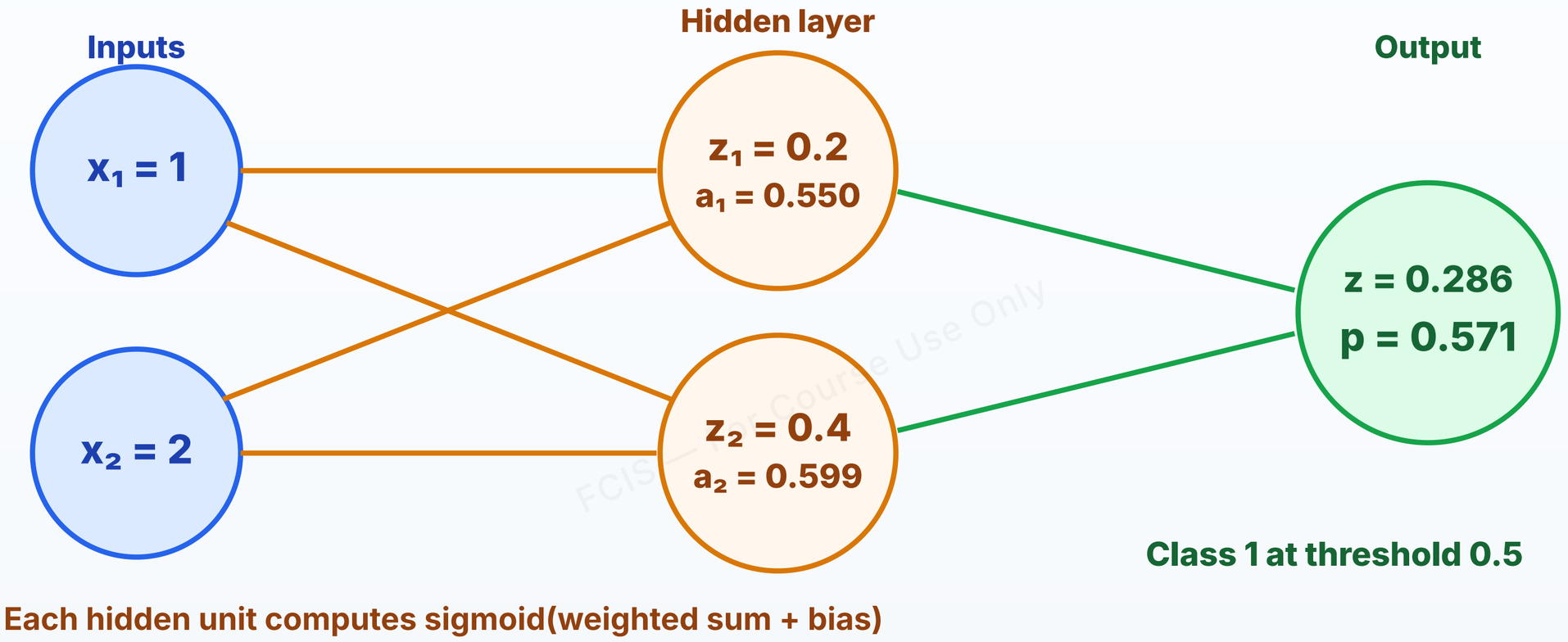
Input → hidden sigmoid units → output probability



FCIS — For Course Use Only

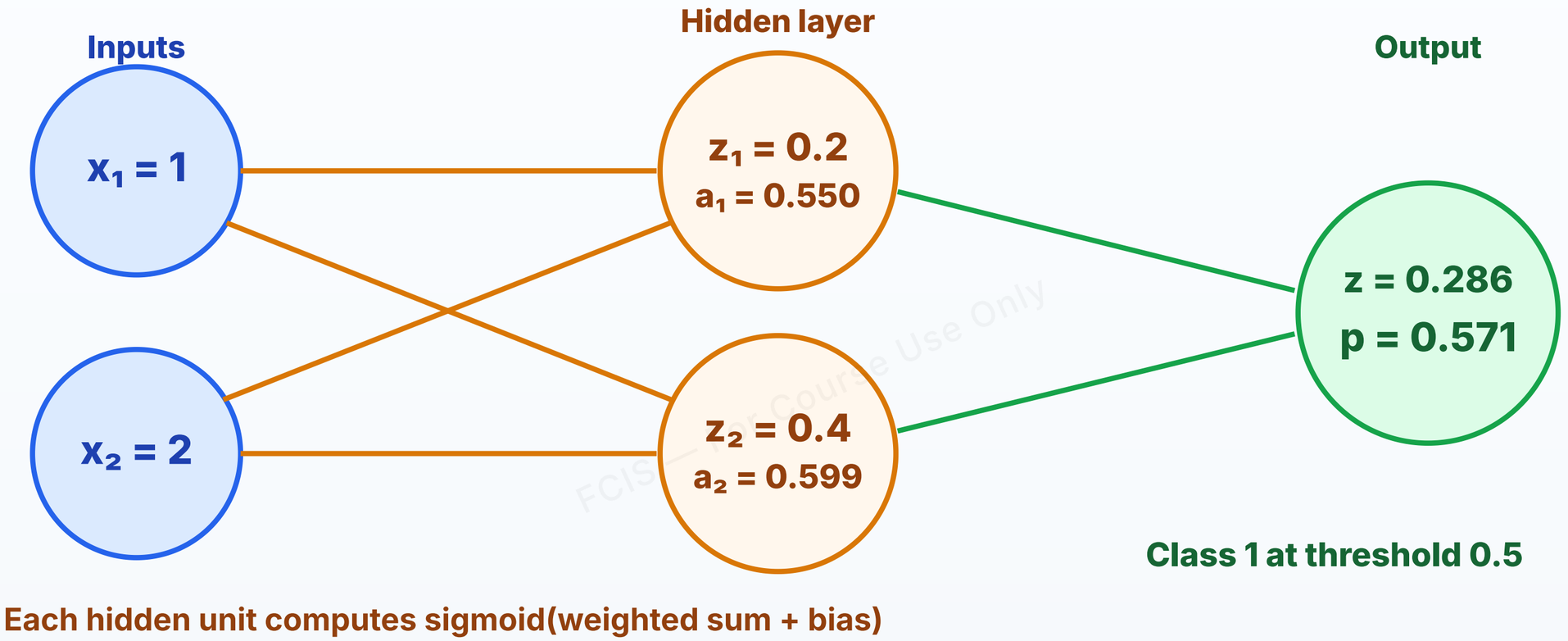
Each hidden unit computes sigmoid(weighted sum + bias)

The same worked network, revealed layer by layer



FCIS — For Course Use Only

The same worked network, revealed layer by layer



FCIS — For Course Use Only

💡 **Roadmap.** The next three slides compute the quantities shown here step by step: hidden unit 1, hidden unit 2, then the output probability.

Worked Forward Pass

FCIS — For Course Use Only

Network definition

Consider a **2 → 2 → 1 neural network** for binary classification. Both the hidden and output neurons use sigmoid activation.

Input: $x = (x_1, x_2)^\top = (1, 2)^\top$

Hidden layer: two neurons

$$z_1 = 0.5x_1 - 0.2x_2 + 0.1$$

$$z_2 = -0.3x_1 + 0.4x_2 - 0.1$$

Apply the **sigmoid activation** to each:

$$a_1 = \sigma(z_1), \qquad a_2 = \sigma(z_2)$$

Output layer: one neuron

$$z_o = 0.7a_1 - 0.5a_2 + 0.2$$

Convert the score into a probability:

$$p = \sigma(z_o)$$

Here, p is the predicted probability of **class 1**.

Next: Compute the hidden activations, then use them to obtain the output probability.

CIS — For Course Use Only

Hidden layer

Substitute the input values $x_1 = 1$ and $x_2 = 2$ into each hidden neuron.

Hidden neuron 1

$$\begin{aligned} z_1 &= 0.5(1) - 0.2(2) + 0.1 \\ &= 0.2 \end{aligned}$$

$$a_1 = \sigma(0.2) \approx 0.550$$

Hidden neuron 2

$$\begin{aligned} z_2 &= -0.3(1) + 0.4(2) - 0.1 \\ &= 0.4 \end{aligned}$$

$$a_2 = \sigma(0.4) \approx 0.599$$

ECIS — For Course Use Only

Hidden representation

The two activated values form the input to the output layer:

$$a^{(1)} = \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} \approx \begin{bmatrix} 0.550 \\ 0.599 \end{bmatrix}$$

Output layer and prediction

1. Compute the output score

Using the hidden activations $a_1 \approx 0.550$ and $a_2 \approx 0.599$:

$$\begin{aligned} z_o &= 0.7a_1 - 0.5a_2 + 0.2 \\ &\approx 0.7(0.550) - 0.5(0.599) + 0.2 \\ &= 0.385 - 0.2995 + 0.2 \\ &= 0.2855 \end{aligned}$$

2. Apply the sigmoid activation

$$p = \sigma(z_o) \approx \sigma(0.2855) \approx 0.571$$

3. Apply the classification threshold

With a threshold of 0.5:

$$\hat{y} = \mathbb{1}(p \geq 0.5) = 1$$

Prediction. The network assigns approximately **57.1% probability** to class 1. Since $0.571 > 0.5$, the predicted class is **1**.

FCIS Prof. Course Use Only

Forward Pass as Composition

FCIS — Course Use Only

The network is one function

A neural network combines several simple transformations into **one composed function**:

$$p = \sigma\left(W^{(2)}\phi\left(W^{(1)}x + b^{(1)}\right) + b^{(2)}\right)$$

Inside: Hidden representation

$$a^{(1)} = \phi(W^{(1)}x + b^{(1)})$$

The hidden layer transforms the original inputs into **learned features**.

Outside: Output prediction

$$p = \sigma(W^{(2)}a^{(1)} + b^{(2)})$$

The output layer converts the hidden representation into a **predicted probability**.

FCIS — For Course Use Only

💡 **Key idea.** Nonlinear hidden activations allow neural networks to model complex relationships. Without them, stacked linear transformations collapse into a single affine transformation.

Training adjusts the weights and biases throughout this composed function to improve predictions.

Part V

FCIS — For Private Use Only

Loss Functions for Neural Networks

The **loss function** measures prediction errors and provides the objective that guides neural-network training.

FCIS — For Course Use Only

Regression Loss

FCIS — Course Use Only

Predicting continuous values

For regression with an **unrestricted numerical target**, the output layer typically uses a **linear activation**:

$$\hat{y} = z_o$$

A common loss is **mean squared error (MSE)**:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

MSE penalizes **larger prediction errors more heavily** because errors are squared.

Example applications

- Predicting rent or property prices.
- Estimating delivery times.
- Forecasting energy demand.
- Predicting exam scores.

FCIS — For Course Use Only

💡 **Key idea.** For unrestricted continuous predictions, use a **linear output** with a suitable regression loss. A sigmoid would unnecessarily restrict predictions to $(0, 1)$.

Binary Classification Loss

FCIS — For Internal Use Only

Sigmoid and binary cross-entropy

For binary classification, the output layer predicts the probability of **class 1**:

$$p = \sigma(z_o)$$

The standard loss for one observation is **binary cross-entropy**:

$$L(y, p) = -\left[y \log(p) + (1 - y) \log(1 - p)\right]$$

Connection to logistic regression

With sigmoid output and binary cross-entropy:

FCIS — For Course Use Only

$$\frac{\partial L}{\partial z_o} = p - y$$

The gradient depends on the difference between the **predicted probability** and the **true label**.

> 💡 **Key idea.** A neural network with a sigmoid output uses the **same loss and output error signal** as binary logistic regression.

Multiclass Classification Loss

FCIS — Course Use Only

Softmax and cross-entropy

For K **mutually exclusive classes**, the output layer produces one score z_k per class.

Softmax converts the scores into probabilities:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

These probabilities **sum to 1**.

Cross-entropy loss

If the correct class is c, the loss is:

$$L = -\log(p_c)$$

FCIS — For Course Use Only

- **High p_c:** Small loss.
- **Low p_c:** Large loss, especially when confidently wrong.

💡 **Key idea.** Softmax produces a **probability distribution across classes**, while cross-entropy encourages the network to assign high probability to the **correct class**.

Part VI

FCIS — For Class Use Only

Backpropagation Intuition

Backpropagation tells each parameter how changing it would change the loss.

FCIS — For Course Use Only

What Backpropagation Solves

FCIS — Course Use Only

Many parameters, one loss

A neural network may contain thousands or millions of parameters.

Training needs gradients such as:

$$\frac{\partial L}{\partial w}$$

Backpropagation efficiently computes all these gradients by applying the chain rule from the output layer backward.

Core question

For each weight:

> If this weight changes slightly, how will the loss change?

> Backpropagation is not a separate learning rule. It is an efficient way to compute gradients for **gradient descent**.

FCIS — For Course Use Only

Chain Rule

FCIS — Students Use Only

The mathematical engine

Backpropagation uses the chain rule to measure how each parameter affects the loss.

A weight influences the loss through a sequence of computations:

$$w \longrightarrow z \longrightarrow a \longrightarrow L$$

Apply the chain rule

$$\frac{\partial L}{\partial w} = \underbrace{\frac{\partial L}{\partial a}}_{\text{loss sensitivity}} \cdot \underbrace{\frac{\partial a}{\partial z}}_{\text{activation derivative}} \cdot \underbrace{\frac{\partial z}{\partial w}}_{\text{weight contribution}}$$

FCIS — For Course Use Only

For a neuron with $z = wx + b$ and $a = \phi(z)$:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a}\, \phi'(z)\, x$$

💡 **Key idea.** Backpropagation computes gradients by **multiplying local derivatives** along the computational path from a parameter to the loss.

Backward Flow

FCIS — Private Use Only

Error signal moves backward

A neural network uses the same computational dependencies in two directions.

Forward pass — compute values

$$x \rightarrow z^{(1)} \rightarrow a^{(1)} \rightarrow z^{(2)} \rightarrow p \rightarrow L$$

Backward pass — compute gradients

$$L \rightarrow p \rightarrow z^{(2)} \rightarrow a^{(1)} \rightarrow z^{(1)} \rightarrow W^{(1)}$$

How the error is assigned:

- **Output layer:** Computes an error signal from the prediction and target.
- **Hidden layers:** Receive error signals through their contributions to subsequent layers.
- **Parameters:** Receive gradients that quantify their effect on the loss.

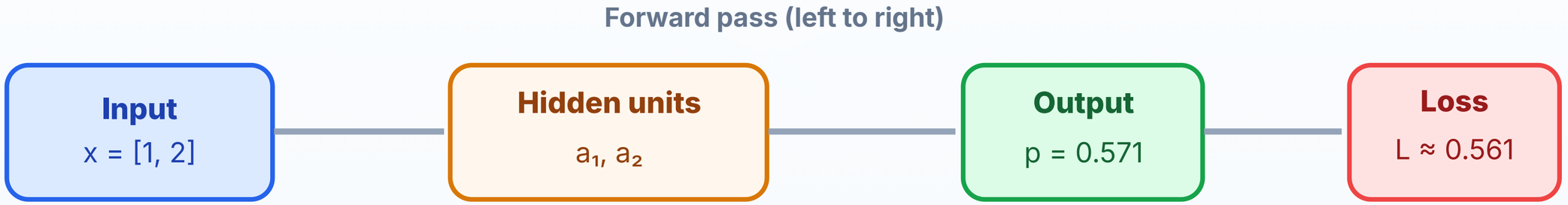
Important. The forward pass computes predictions; backpropagation computes **gradients**. The optimizer then uses these gradients to update the weights and biases.

Visual Walkthrough: Backpropagation

For Course Use Only

Gradients travel backward through the same network

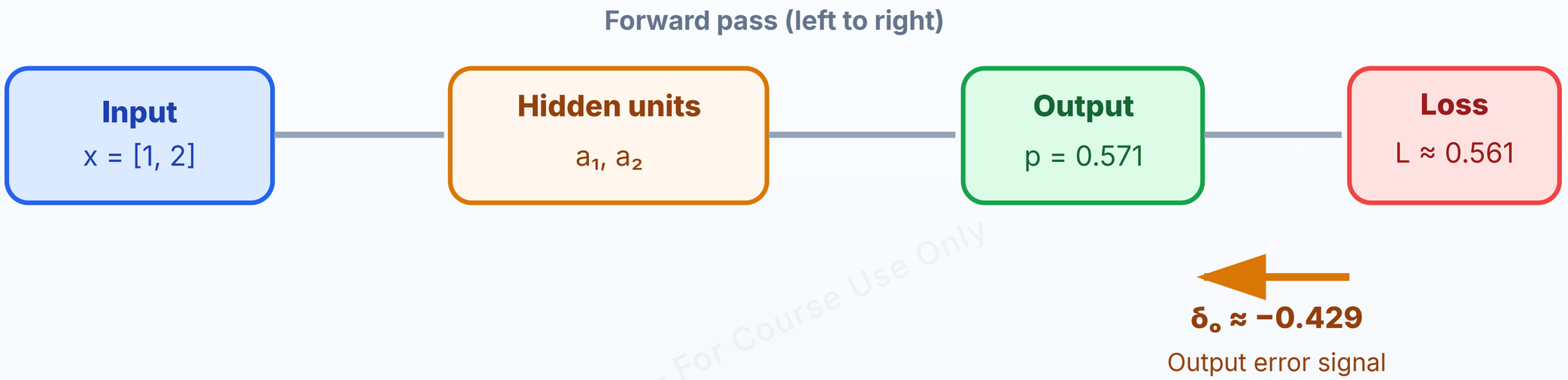
Forward values are known; gradients move right to left



FCIS — For Course Use Only

Gradients travel backward through the same network

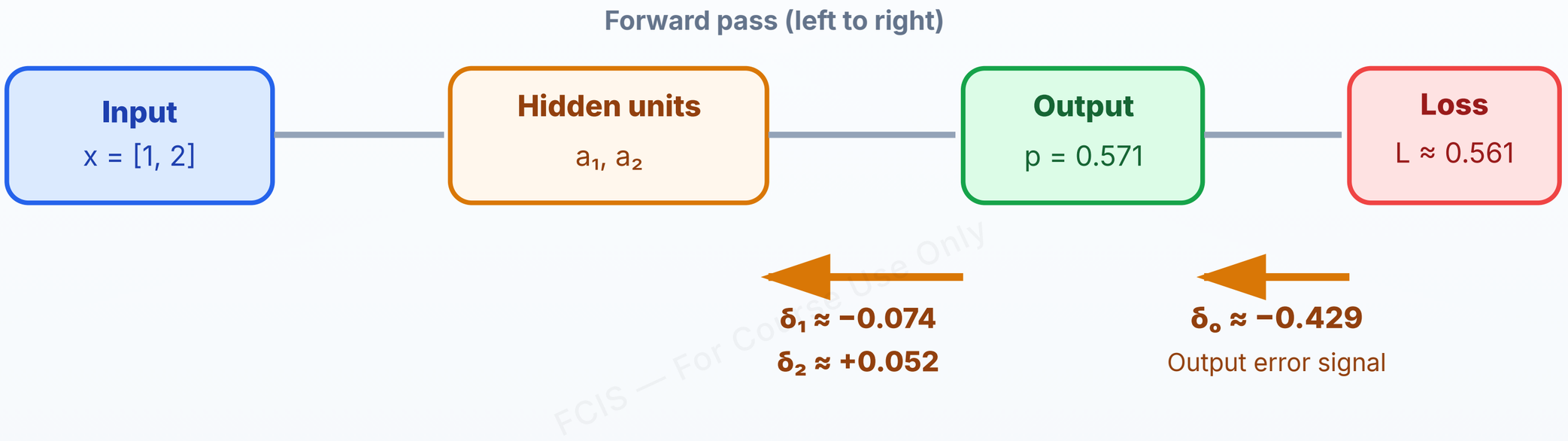
Forward values are known; gradients move right to left



FCIS — For Course Use Only

Gradients travel backward through the same network

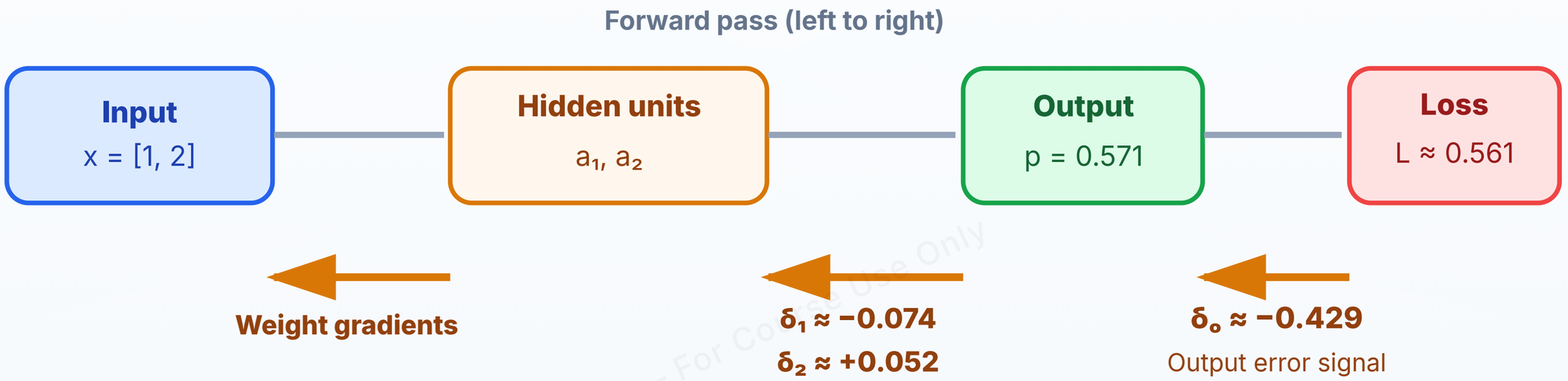
Forward values are known; gradients move right to left



FCIS — For Course Use Only

Gradients travel backward through the same network

Forward values are known; gradients move right to left

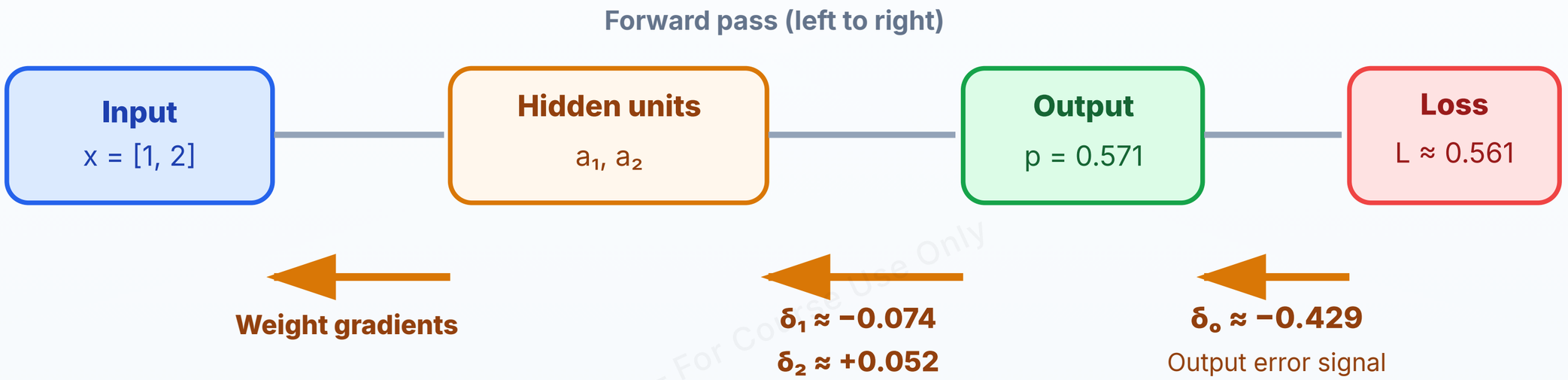


FCIS — For Course Use Only

Compute gradients using the original forward-pass weights

Gradients travel backward through the same network

Forward values are known; gradients move right to left



FCIS — For Course Use Only

Compute gradients using the original forward-pass weights

💡 **Key idea.** With target $y = 1$, the output error signal is $\delta_o = p - y \approx -0.429$. Hidden units receive their error signals through downstream weights and activation derivatives. **Compute all gradients before updating parameters.**

Output Layer Gradient

FCIS — Private Use Only

Binary classification case

For a **sigmoid output** with **binary cross-entropy loss**, the output score is $z_o = v_1 a_1 + v_2 a_2 + b_o$.

The **output error signal** simplifies to:

$$\boxed{\delta_o = \frac{\partial L}{\partial z_o} = p - y}$$

Parameter gradients

$$\frac{\partial L}{\partial v_1} = \delta_o a_1$$

$$\frac{\partial L}{\partial v_2} = \delta_o a_2$$

$$\frac{\partial L}{\partial b_o} = \delta_o$$

Worked example

With $p \approx 0.571$ and $y = 1$:

$$\delta_o = 0.571 - 1 \approx -0.429$$

The **negative error signal** means that increasing the output score would reduce the loss for this example.

FCIS — For Course Use Only

Hidden Layer Gradient

FCIS — Use Only

Assign responsibility to hidden units

A hidden neuron’s error depends on the **downstream error**, its **output connection weight**, and its **activation derivative**.

Hidden error signal

For hidden neuron j:

$$\begin{aligned}\delta_j &= \frac{\partial L}{\partial z_j} \\ &= \delta_o v_j \, \phi'(z_j)\end{aligned}$$

Here, $\delta_o v_j$ is the **downstream signal**, and $\phi'(z_j)$ is the **local derivative**.

Parameter gradients

For $z_j = w_{j1}x_1 + w_{j2}x_2 + b_j$:

$$\frac{\partial L}{\partial w_{j1}} = \delta_j x_1$$

$$\frac{\partial L}{\partial w_{j2}} = \delta_j x_2$$

$$\frac{\partial L}{\partial b_j} = \delta_j$$

ECIS — For Course Use Only

💡 **Key idea.** The hidden error signal combines the **downstream error**, **connection weight**, and **activation derivative**. Multiplying it by each input gives the corresponding weight gradient.

Sigmoid Derivative

FCIS — Your Course Use Only

Needed in the hidden layer

For a sigmoid neuron, $a = \sigma(z)$.

The derivative can be computed directly from the **activation output**:

$$\boxed{\sigma'(z) = \sigma(z)(1 - \sigma(z)) = a(1 - a)}$$

Apply it to our two hidden neurons

Hidden neuron	Activation	Local derivative
1	$a_1 \approx 0.550$	$0.550(1 - 0.550) = 0.2475$
2	$a_2 \approx 0.599$	$0.599(1 - 0.599) \approx 0.240$

CIS — For Course Use Only

These derivatives are used to calculate δ_1 and δ_2 during backpropagation.

Interpretation. Sigmoid derivatives are largest near an activation of 0.5. When outputs approach **0 or 1**, derivatives become small, potentially weakening gradient flow.

ReLU Derivative

FCIS — For Educational Use Only

Piecewise derivative

ReLU passes positive inputs through unchanged and replaces negative inputs with zero.

$$\mathrm{ReLU}(z) = \max(0, z)$$

Its derivative is:

$$\mathrm{ReLU}'(z) = \begin{cases} 0, & z < 0 \\ 1, & z > 0 \end{cases}$$

At $z = 0$, the derivative is undefined; implementations use a convention, commonly **0**.

FCIS — For Course Use Only

Examples

Pre-activation	ReLU output	Local derivative
$z = -2$	0	0
$z = 3$	3	1

💡 **Gradient effect.** For **positive** inputs, ReLU passes the local gradient through. For **negative** inputs, the local gradient is zero, so that neuron receives no gradient contribution from that example through its ReLU.

Part VII

FCIS — Students Use Only

Full Backpropagation Worked Example

One complete training step

We will use the **same network** to compute error signals, gradients, parameter updates, and a new prediction.

FCIS — For Course Use Only

Worked Example Setup

FCIS — Academic Use Only

Network and label

We use the **2 → 2 → 1 network** from the forward-pass example.

Input and target: $x = (1, 2)^\top, \qquad y = 1$

Forward-pass results

- **Hidden neuron 1:** $a_1 \approx 0.550$
- **Hidden neuron 2:** $a_2 \approx 0.599$
- **Output probability:** $p \approx 0.571$

For $y = 1$, the binary cross-entropy loss is:

$$L = -\log(p) \approx 0.561$$

Parameters and training settings

Output-layer parameters:

$$v_1 = 0.7, \quad v_2 = -0.5, \quad b_o = 0.2$$

Learning rate:

$$\alpha = 0.1$$

We will compute gradients for **all parameters** before applying the updates.

FCIS - For Course Use Only

Output Error Signal

FCIS — Private Use Only

Start from the prediction error

For a **sigmoid output** with **binary cross-entropy loss**, the output error signal simplifies to:

$$\boxed{\delta_o = \frac{\partial L}{\partial z_o} = p - y}$$

Substitute the worked-example values

$$\begin{aligned} \delta_o &= 0.571 - 1 \\ &\approx -0.429 \end{aligned}$$

FCIS — For Course Use Only

Interpretation

- The target is **1**, but the model predicts only **0.571**.
- The negative error signal indicates that **increasing the output score** would reduce the loss for this example.
- This signal is used to compute the output and hidden-layer gradients.

Output Layer Gradients

FCIS — For Course Use Only

Gradients for v_1, v_2, and b_o

Using the output error signal $\delta_o \approx -0.429$:

$$\frac{\partial L}{\partial v_j} = \delta_o a_j, \qquad \frac{\partial L}{\partial b_o} = \delta_o$$

Calculate each gradient

Parameter	Calculation	Gradient
v_1	$(-0.429)(0.550)$	≈ -0.236
v_2	$(-0.429)(0.599)$	≈ -0.257
b_o	$\delta_o = -0.429$	-0.429

For Course Use Only

Interpretation: All three gradients are negative. Gradient descent will therefore **increase** these output parameters.

The gradients are recorded now; the updates are applied after computing the hidden-layer gradients.

Hidden Error Signals

FCIS — Course Use Only

Backpropagate through output weights

Use the **original output weights** $v_1 = 0.7$ and $v_2 = -0.5$, together with $\delta_o \approx -0.429$.

For sigmoid hidden neurons:

$$\delta_j = \delta_o v_j a_j (1 - a_j)$$

Hidden neuron 1

$$\delta_1 = \delta_o v_1 a_1 (1 - a_1)$$

$$\approx (-0.429)(0.7)$$
$$(0.550)(0.450)$$

$$\approx \boxed{-0.074}$$

Hidden neuron 2

$$\delta_2 = \delta_o v_2 a_2 (1 - a_2)$$

$$\approx (-0.429)(-0.5)$$
$$(0.599)(0.401)$$

$$\approx \boxed{+0.052}$$

FCIS — For Course Use Only

Key observation: The two hidden error signals have **opposite signs** because the output connection weights have opposite signs.

Hidden Layer Gradients

FCIS — Academic Use Only

Gradients for input-to-hidden weights

For a hidden neuron j, the parameter gradients are:

$$\frac{\partial L}{\partial w_{jk}} = \delta_j x_k, \qquad \frac{\partial L}{\partial b_j} = \delta_j$$

Using $x_1 = 1$, $x_2 = 2$, $\delta_1 \approx -0.074$, and $\delta_2 \approx 0.052$:

Hidden neuron	Gradient	Calculation and result
Neuron 1	$\partial L/\partial w_{11}$	$(-0.074)(1) = -0.074$
	$\partial L/\partial w_{12}$	$(-0.074)(2) = -0.148$
	$\partial L/\partial b_1$	-0.074
Neuron 2	$\partial L/\partial w_{21}$	$(0.052)(1) = 0.052$
	$\partial L/\partial w_{22}$	$(0.052)(2) = 0.104$
	$\partial L/\partial b_2$	0.052

CIS — For Course Use Only

Now all nine parameter gradients are available: three for the output layer and six for the hidden layer.

Output Layer Update

FCIS — Course Use Only

Apply gradient descent

With learning rate $\alpha = 0.1$, update each parameter using:

$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \frac{\partial L}{\partial \theta}$$

Output-layer updates

Parameter	Update calculation	New value
v_1	$0.7 - 0.1(-0.236)$	**0.7236**
v_2	$-0.5 - 0.1(-0.257)$	**−0.4743**
b_o	$0.2 - 0.1(-0.429)$	**0.2429**

For Course Use Only

The updated output-layer parameters are:

$$v_1^{\text{new}} = 0.7236, \quad v_2^{\text{new}} = -0.4743, \quad b_o^{\text{new}} = 0.2429$$

Interpretation. Gradient descent adjusts the output parameters in directions that should increase the correct-class probability for this training example.

Hidden Layer Update

FCIS — Private Use Only

Apply gradient descent

Apply the **same learning rate**, $\alpha = 0.1$, to the hidden-layer parameters using the gradients already computed.

Hidden neuron 1

Parameter	Update calculation	New value
w_{11}	$0.5 - 0.1(-0.074)$	**0.5074**
w_{12}	$-0.2 - 0.1(-0.148)$	**−0.1852**
b_1	$0.1 - 0.1(-0.074)$	**0.1074**

Hidden neuron 2

Parameter	Update calculation	New value
w_{21}	$-0.3 - 0.1(0.052)$	**−0.3052**
w_{22}	$0.4 - 0.1(0.104)$	**0.3896**
b_2	$-0.1 - 0.1(0.052)$	**−0.1052**

FCIS — For Course Use Only

Important: Both layers are updated using gradients computed from the **same original forward pass**. The calculations use rounded gradient values.

What Did Backpropagation Do?

FCIS — Private Use Only

Interpret the update

For a training example with **true label** $y = 1$, the network initially predicted $p \approx 0.571$.

The training step involved three operations:

1. Forward propagation

Computed the hidden activations, output probability, and loss.

2. Backpropagation

Computed how **each weight and bias affected the loss**, using the chain rule and the original parameters.

3. Gradient descent

Updated the weights and biases using:

$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \nabla_\theta L$$

Key distinction. Backpropagation **computes gradients**; gradient descent **uses those gradients** to update the parameters. The next forward pass tells us whether the update helped.

FCIS — For Course Use Only

Worked Example Check: Did the Update Help?

For Course Use Only

Recompute the output after updating every parameter

Using all the updated weights and biases, perform a **new forward pass**.

Updated hidden activations

$$a_1^{\text{new}} \approx 0.561, \qquad a_2^{\text{new}} \approx 0.591$$

Updated output probability

$$z_o^{\text{new}} \approx 0.368, \qquad p^{\text{new}} = \sigma(z_o^{\text{new}}) \approx 0.591$$

Before vs. after one training step

Quantity	Before update	After update
Class-1 probability	0.571	**0.591**
Loss $-\log(p)$	0.561	**0.526**

For this example, the predicted probability of the correct class **increased**, while the loss **decreased**.

> **Takeaway.** One gradient step improved the loss on this **single training example**. It does not establish that the network will generalize; validation performance must still be evaluated.

FCIS — For Course Use Only

Part VIII

FCIS — Lecture Use Only

Training Neural Networks

Training a **neural network** means repeatedly computing predictions, measuring loss, calculating gradients, and updating parameters.

FCIS — For Course Use Only

Training Loop

FCIS — Course Use Only

High-level algorithm

Neural-network training repeats the following procedure over multiple **epochs**:

```
initialize weights and biases

for each epoch:
    shuffle the training examples
    divide them into mini-batches

    for each mini-batch:
        forward pass → compute predictions
        compute loss
        backward pass → compute gradients
        optimizer step → update parameters

    evaluate training and validation performance
```

FCIS — For Course Use Only

Important distinction

- **Parameters** (weights and biases) are learned from **training data**.
- **Hyperparameters** (such as learning rate and batch size) are selected using **validation evidence**.
- Validation examples are **not used for gradient updates**.

Epochs and Batches

FCIS — Students Use Only

Training terminology

Term	Meaning
Epoch	One complete pass through the training dataset
Batch	Group of training examples processed together
Mini-batch	Relatively small batch, commonly used in training
Iteration	One optimizer update of the parameters
Learning rate	Controls the size of parameter updates

CCIS — For Course Use Only

Example

Suppose we have **1,000 training examples** and a mini-batch size of **100**.

$$\text{Updates per epoch} = \frac{1000}{100} = 10$$

One epoch = 10 parameter updates. Training for 20 epochs therefore performs **200 updates**, assuming the same batch size and no gradient accumulation.

Weight Initialization

FCIS — Personal Use Only

Starting values matter

Neural-network **weights** are generally initialized with small, non-identical random values.

Why not initialize every weight to zero?

If neurons in the same layer start with identical weights and biases, they receive identical gradients and may learn the **same features**.

Random initialization breaks this symmetry.

Good initialization helps

- maintain stable gradients;
- speed up optimization;
- make effective use of hidden units;
- reduce saturation problems.

Common approaches

- **Xavier/Glorot:** Often used with tanh-like activations.
- **He initialization:** Commonly used with ReLU-like activations.
- **Biases:** Can often start at zero because random weights already break symmetry.

FCIS — For Course Use Only

💡 **Key idea.** Initialization affects how easily a network can be trained. It is an important part of **optimization**, not merely a coding detail.

Learning Rate

FCIS — For Course Use Only

Too small or too large

The **learning rate** α controls how far parameters move during gradient descent:

$$\boxed{\theta_{\text{new}} = \theta_{\text{old}} - \alpha \nabla_\theta L}$$

Learning rate	Possible training behavior
Too small	Updates are tiny; training progresses slowly
Appropriate	Loss often decreases efficiently and relatively smoothly
Too large	Updates may overshoot, oscillate, or diverge

FCIS — For Course Use Only

Why monitor the loss?

- A slowly decreasing loss may indicate a learning rate that is too small.
- An unstable or increasing loss may indicate a learning rate that is too large.
- A suitable learning rate helps, but **does not guarantee good generalization**.

💡 **Practical lesson.** Training curves help diagnose whether optimization is progressing and whether the learning rate needs adjustment.

Training and Validation Curves

FCIS — Personal Use Only

Diagnose learning behavior

Compare the **training loss** and **validation loss** across epochs.

Observed pattern	Possible interpretation
Both losses remain high	Underfitting or ineffective optimization
Training loss low, validation loss high	Overfitting
Both losses decrease and stabilize	Learning is progressing
Training loss fluctuates strongly	Learning rate may be too high
Validation loss starts increasing	Further training may increase overfitting

ECIS — For Cou... Use O...

What should we monitor?

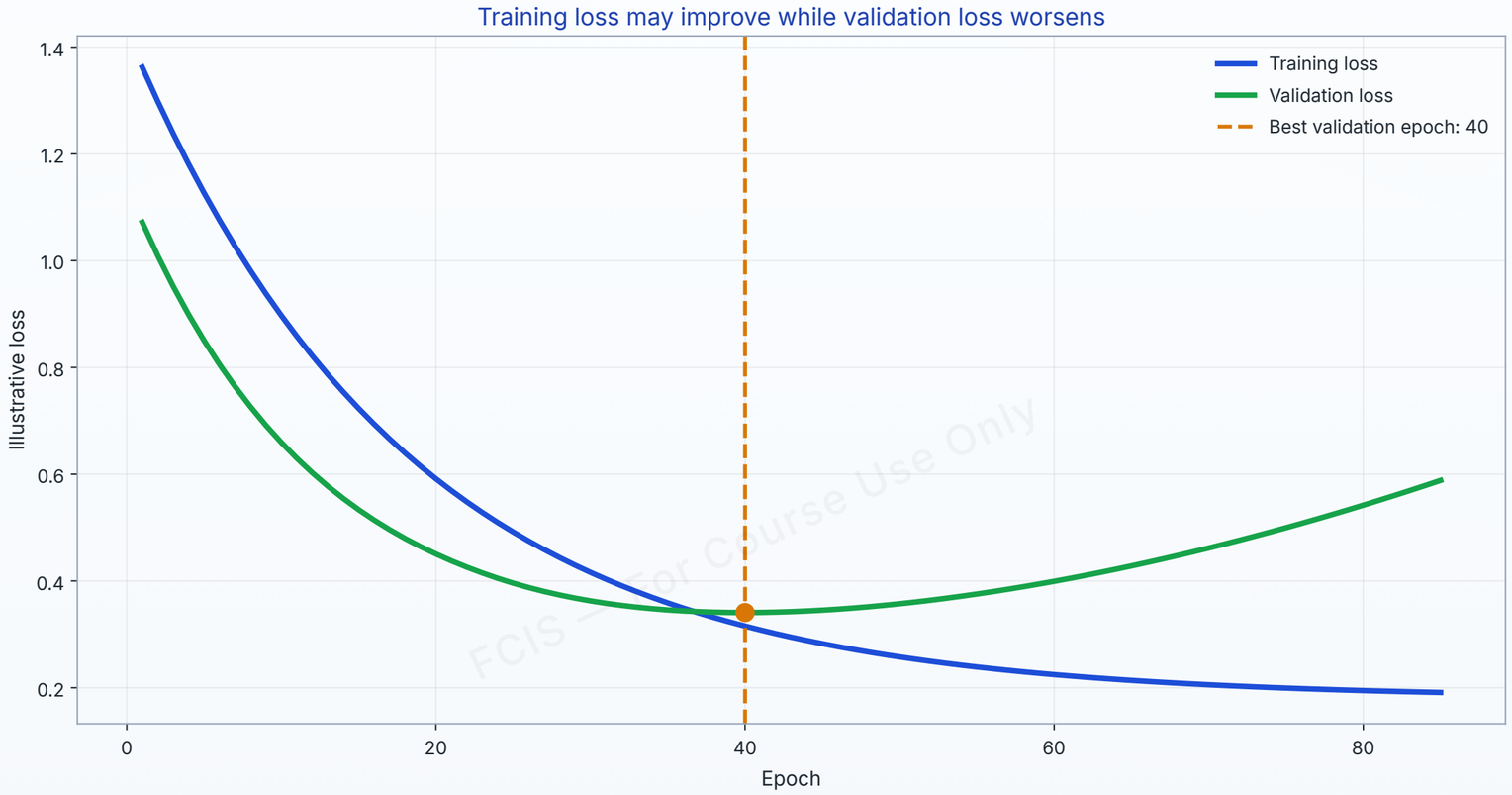
- Training loss shows how well the network fits the **training examples**.
- Validation loss estimates performance on **held-out data**.
- The epoch with the lowest validation loss may occur **before the final epoch**.

💡 **Takeaway.** Monitor neural networks throughout training. Select models using **validation evidence**, rather than assuming the last epoch is best.

Plot: Training Curves and Early Stopping

For Course Only

Why the best epoch is not always the last epoch



Key idea. The curves are **illustrative**, not measured results. Early stopping monitors validation loss and may stop after several epochs without improvement (*patience*), restoring the weights from the best validation epoch.

Vanishing Gradients

FCIS — Personal Use Only

Gradients become too small

Backpropagation repeatedly applies the **chain rule**. In a deep network, multiplying many small local derivatives can make gradients extremely small.

This can occur with saturating activations such as **sigmoid** and **tanh**.

What can happen?

- Early layers receive **tiny gradients**.
- Their weights change very slowly.
- Optimization becomes difficult.
- Adding layers may not improve performance.

Common responses

- Use **ReLU-like activations**.
- Choose suitable weight initialization.
- Normalize input features.
- Use architectures such as residual networks and suitable optimization methods.

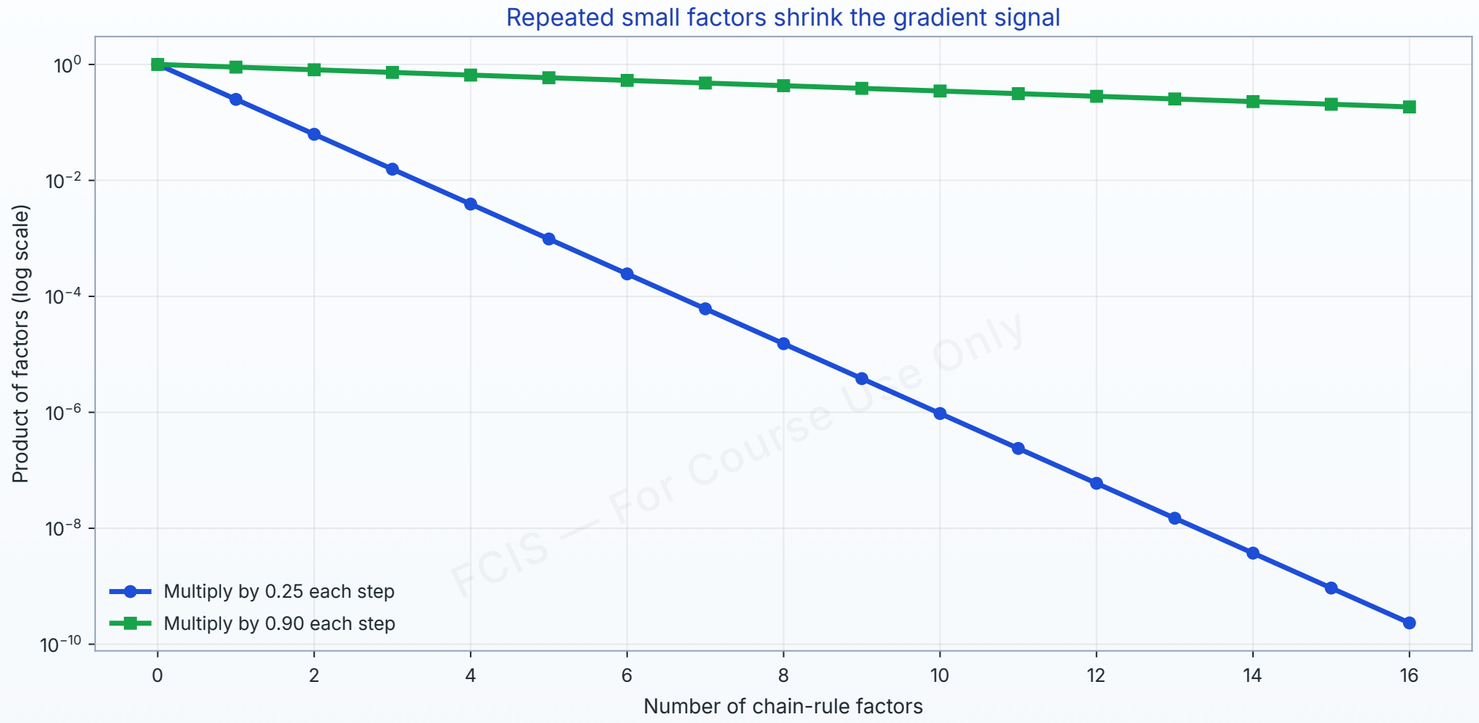
FCIS - For Course Use Only

💡 **Key idea.** Vanishing gradients make it difficult for early layers to learn because the training signal weakens as it propagates backward.

Plot: Vanishing Gradients

FCIS — Personal Use Only

What happens when small derivatives are repeatedly multiplied?



FCIS — For Course Use Only

Interpretation. This is a **toy multiplication example**, not measured network gradients. A sigmoid derivative is at most 0.25, so repeated small factors can shrink a gradient rapidly. Real gradient flow also depends on **weights, activations, initialization, and network architecture**.

Exploding Gradients

FCIS — For Educational Use Only

Gradients become too large

The chain rule can also cause gradients to **grow excessively**, especially when repeated transformations amplify the backward signal.

Possible symptoms

- Parameter updates become very large.
- Training loss becomes unstable.
- Loss may increase dramatically.
- Parameters may diverge or become numerically invalid.

Common responses

- Reduce the **learning rate**.
- Use **gradient clipping**.
- Improve weight initialization.
- Normalize input features.
- Monitor gradients and training curves.

FCIS — For Course Use Only

💡 **Key distinction. Vanishing gradients** make learning extremely slow; **exploding gradients** make optimization unstable. Both are training problems that require appropriate optimization and network design.

Part IX

FCIS — Personal Use Only

Overfitting and Regularization

Neural networks can fit complex patterns, including noise. Regularization helps control model complexity and improve generalization.

FCIS — For Course Use Only

Why Neural Networks Overfit

FCIS — Private Use Only

Many parameters can memorize data

Neural networks can have enough capacity to fit **training noise** rather than patterns that generalize.

Common risk factors

- network too large for the dataset;
- training continues too long;
- noisy training labels;
- insufficient regularization.

Evaluation pitfalls

- features contain **data leakage**;
- validation performance is ignored.

Data leakage is a separate problem: it can make reported performance **misleadingly optimistic**.

FCIS — For Course Use Only

Typical symptom of overfitting

Training loss keeps decreasing, while **validation loss starts increasing**.

Key lesson. A network that memorizes training examples may achieve very low training loss but perform poorly on unseen data.

Early Stopping

FCIS — For Course Use Only

Stop when validation performance stops improving

Early stopping monitors validation performance during training to decide when to stop.

How it works

1. Train the network for one epoch.
2. Measure the **validation loss**.
3. Save the model weights when validation loss improves.
4. Stop after several epochs without sufficient improvement (*patience*).

FCIS — For Course Use

Why it helps

- prevents unnecessary training;
- can reduce overfitting.

Model selection

- selects a model state using validation evidence;
- often works well in practice.

💡 **Key idea.** Early stopping treats training duration as a **hyperparameter**. Restore the **best validation weights**, not necessarily the weights from the final epoch.

Weight Decay

FCIS Internal Use Only

Penalize large weights

A common approach to regularization adds an L_2 **penalty** to the training objective:

$$L_{\text{total}} = L_{\text{data}} + \lambda \sum_j w_j^2$$

Here, $\lambda \geq 0$ controls the **regularization strength**.

Why penalize large weights?

- discourages excessively large weights;
- limits some overly complex solutions;
- can improve generalization;
- is similar in spirit to **Ridge regression**.

FCIS — For Course Use Only

Weight decay often refers to directly shrinking weights during optimization. Its equivalence to an L_2 penalty depends on the optimizer.

Practical lesson. Stronger regularization is not always better: excessive regularization can cause **underfitting**. Select its strength using validation data.

Dropout

FCIS — For Course Use Only

Randomly deactivate units during training

Dropout randomly sets selected hidden activations to zero during training.

Why use it?

The network is encouraged to learn representations that do not depend too heavily on any single hidden unit.

During training	During evaluation
Randomly deactivate some units	Use all units
Different units may be active in each step	No random dropout
Apply appropriate activation scaling	Use the corresponding evaluation scaling

FCIS — For Course Use Only

Potential benefits

- encourages more robust representations;
- reduces excessive co-adaptation;
- can improve **generalization**.

💡 **Key idea.** Dropout introduces randomness **during training**, not ordinary prediction. With common *inverted dropout*, scaling is applied during training so no additional scaling is needed at evaluation.

Data Augmentation

FCIS — Course Use Only

More varied training examples

Data augmentation creates additional training examples through transformations that should **preserve the correct label**.

Examples

Data type	Possible augmentation
Images	Small rotations, shifts, crops
Audio	Small noise additions or time shifts
Text	Carefully controlled paraphrasing or synonym replacement
Tabular data	Domain-specific transformations, only when justified

FCIS — For Course Use Only

Why it helps

- increases variation in the training data;
- encourages robustness to realistic changes;
- can reduce overfitting.

Important. An augmentation is valid only when it preserves the intended label. Apply augmentation to **training data**, keeping validation and test examples suitable for unbiased evaluation.

Part X

FCIS — For Private Use Only

Preprocessing for Neural Networks

Neural networks learn from numerical representations. Appropriate scaling and encoding can make training more effective and reliable.

FCIS — For Course Use Only

Scaling Inputs

FCIS — Course Use Only

Help optimization

Neural networks often train more effectively when numerical features have **comparable scales**.

Common scaling methods

Standardization

$$z = \frac{x - \mu}{\sigma}$$

Centers a feature around zero and scales it by its standard deviation.

Min–max scaling

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Maps the training feature range to $[0, 1]$ when the range is nonzero.

FCIS — For Course Use Only

Why scaling helps

- gradients can become more stable;
- learning-rate selection becomes easier;
- optimization may converge faster;
- features are less likely to dominate solely because of their units.

Prevent data leakage. Fit the scaler using **training data only**. Apply the same learned transformation to validation, test, and future inputs.

Encoding Inputs

FCIS — Private Use Only

Categorical variables need representation

Neural networks require **numerical inputs**. The encoding should match the type of categorical feature.

Common encoding choices

- **Nominal:** One-hot encoding avoids artificial ordering.
- **High-cardinality:** Learned embeddings provide compact representations.
- **Ordinal:** Preserve meaningful category order.
- **Unseen categories:** Define an unknown-category strategy.

Example: One-hot encoding

For **Study Mode** with three categories:

Category	Online	In-person	Hybrid
Online	1	0	0
In-person	0	1	0
Hybrid	0	0	1

For — For Course Use Only

💡 **Key idea.** Encoding determines how the network represents categories. Arbitrary numeric category codes may introduce an ordering that does not exist.

Output Scaling

FCIS — Internal Use Only

Especially important for regression

For **neural-network regression**, the target scale can affect the **loss magnitude**, gradients, and optimization.

A target with very large numerical values can produce a large mean squared error (MSE).

Possible strategies

Strategy	When it may help
Target standardization	Targets have large magnitudes or inconvenient scales
Log transformation	Positive targets are strongly right-skewed
Inverse transformation	Converts model predictions back to meaningful units
Evaluation in original units	Makes errors interpretable for the application

FCIS — For Course Use Only

Example

If a network predicts **standardized house prices**, the output must be converted back to the original currency before reporting prediction errors.

Key idea. Fit any target transformation on **training targets only**. Invert it when producing predictions in original units, and report evaluation metrics in units meaningful to the application.

Part XI

FCIS — Personal Use Only

Neural Networks in Practice

Neural networks are useful when the data, prediction task, and potential performance gains justify their complexity.

FCIS — For Course Use Only

When Neural Networks Are Useful

FCIS — For Course Use Only

Typical strengths

Neural networks can be particularly effective when the task involves **complex patterns** or requires learning useful representations directly from data.

Common applications

- image recognition;
- speech processing;
- text analysis;
- high-dimensional inputs.

Potential strengths

- modeling complex nonlinear relationships;
- learning from large datasets;
- learning useful feature representations.

FCIS — For Course Use Only

What about tabular data?

Neural networks can also work well with structured tabular data. However, **linear models, decision trees, and tree-based ensembles** often provide strong alternatives.

> 💡 **Key idea.** Use neural networks when their added complexity provides a meaningful benefit. Always compare them with **simpler baseline models** using validation performance.

When Simpler Models May Be Better

FCIS - For Course Use Only

Not every problem needs deep learning

A more complex model is not necessarily a better choice.

Simpler models may be preferable when:

- the dataset is small;
- interpretability is essential;
- features are already well-structured;
- training resources are limited;
- deployment must be fast;
- validation performance is similar.

Example applications

- **Medical tabular data:** Limited training examples.
- **Credit scoring:** Strong interpretability requirements.
- **Linear trend prediction:** A simple relationship may be sufficient.
- **Rule-like decisions:** A compact tree may be easier to explain.

FCIS — For Course Use Only

💡 **Model selection principle.** Choose the simplest model that adequately meets the application's **predictive, computational, and interpretability requirements**.

Interpretability Challenges

FCIS — Private Use Only

Powerful but less transparent

Neural networks may achieve strong predictive performance while remaining difficult to interpret.

Why can interpretation be difficult?

- **Many interacting parameters:** Individual weights rarely have simple explanations.
- **Hidden representations:** Learned features are not always human-readable.
- **Sensitivity to inputs:** Some small changes can produce unexpected predictions.
- **Additional explanation methods:** Understanding predictions may require specialized tools.

Accuracy is not the same as interpretability

A model can produce accurate predictions without providing an explanation that is clear enough for its intended users.

Key distinction. Predictive performance, interpretability, and reliability are **different requirements**. In high-stakes applications, all three may need careful evaluation.

Practical Development Checklist

FCIS — Private Use Only

Before trusting a neural network

Use this checklist before selecting or deploying a neural-network model.

Data and model setup

1. Is the prediction task clearly defined?
2. Are inputs properly scaled and encoded?
3. Is there a strong baseline model?
4. Are training, validation, and test splits correct?
5. Have potential sources of **data leakage** been checked?

Training and evaluation

6. Do training and validation curves make sense?
7. Is **overfitting** adequately controlled?
8. Are evaluation metrics appropriate for the task?
9. Is the model sufficiently stable across runs or splits?
10. Can its predictions be explained sufficiently for the application?

FCIS — For Course Use Only

💡 **Takeaway.** A neural network should be evaluated as a complete **machine-learning system**, not judged only by its training accuracy or loss.

Practice & Consolidation

FCIS — For Course Use Only

Neural-Network Reasoning and Common Mistakes

Strengthen your understanding by tracing **neuron computations**, choosing suitable outputs, diagnosing training, and reasoning about gradients.

FCIS — For Course Use Only

Activity: Neuron Computation

FCIS — Personal Use Only

Manual practice

A neuron has three inputs, three weights, and a bias:

$$w = \begin{bmatrix} 0.2 \\ 0.5 \\ -0.4 \end{bmatrix}, \qquad x = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}, \qquad b = 0.3$$

The neuron uses the **ReLU activation**.

Tasks

1. Compute the weighted sum $z = w^\top x + b$.
2. Compute the activated output $a = \mathrm{ReLU}(z)$.
3. Determine whether the neuron is **active** or **inactive**.

IUG — For Course Use Only

Manual practice

A neuron has three inputs, three weights, and a bias:

$$w = \begin{bmatrix} 0.2 \\ 0.5 \\ -0.4 \end{bmatrix}, \qquad x = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}, \qquad b = 0.3$$

The neuron uses the **ReLU activation**.

Tasks

1. Compute the weighted sum $z = w^\top x + b$.
2. Compute the activated output $a = \mathrm{ReLU}(z)$.
3. Determine whether the neuron is **active** or **inactive**.

IUG — For Course Use Only

Solution.

$$z = 0.2(1) + 0.5(2) - 0.4(1) + 0.3 = 1.1$$
$$a = \mathrm{ReLU}(1.1) = 1.1$$

The neuron is **active** because its pre-activation is positive.

Activity: Choose the Output Layer

FCIS — Educational Use Only

Match task to activation and loss

For each task, choose an appropriate **output activation** and **loss function**.

Prediction task	Output activation	Loss function
Predict house price	?	?
Detect spam or not spam	?	?
Classify digit 0–9	?	?
Predict delivery time	?	?
Predict disease present/absent	?	?

FCIS — For Course Use Only

Match task to activation and loss

For each task, choose an appropriate **output activation** and **loss function**.

Prediction task	Output activation	Loss function
Predict house price	?	?
Detect spam or not spam	?	?
Classify digit 0–9	?	?
Predict delivery time	?	?
Predict disease present/absent	?	?

FCIS — For Course Use Only

Solution.

- **House price / delivery time:** Linear output with MSE or MAE.
- **Spam / disease presence:** Sigmoid output with binary cross-entropy.
- **Digit classification (0–9):** Softmax output with categorical cross-entropy.

Activity: Interpret Training Curves

FCIS — For Students Use Only

Diagnose learning behavior

Assume training and validation losses are measured using the same loss function.

What does each pattern suggest?

Pattern	Training loss	Validation loss	Likely interpretation
A	High and flat	High and flat	?
B	Decreases	Decreases, then increases	?
C	Unstable	Unstable	?
D	Decreases steadily	Decreases steadily	?

FCIS – For Course Use Only

Diagnose learning behavior

Assume training and validation losses are measured using the same loss function.

What does each pattern suggest?

Pattern	Training loss	Validation loss	Likely interpretation
A	High and flat	High and flat	?
B	Decreases	Decreases, then increases	?
C	Unstable	Unstable	?
D	Decreases steadily	Decreases steadily	?

FCIS – For Course Use Only

Solution.

- **A:** Underfitting or an optimization problem.
- **B:** Overfitting begins after some epochs; early stopping may help.
- **C:** The learning rate may be too high, although other issues are possible.
- **D:** Training appears to be progressing; continue monitoring validation performance.

Activity: Backpropagation Reasoning

For Course Use Only

Direction of update

A binary classifier with a **sigmoid output** predicts:

$$p = 0.25, \qquad y = 1$$

Assume training uses **binary cross-entropy loss**.

Questions

1. Is the predicted probability of class 1 too high or too low?
2. Calculate the output error signal $\delta_o = p - y$.
3. Should the output score z_o increase or decrease to reduce this example's loss?
4. Explain your answer using the sign of the gradient.

FCIS _ For Course Use Only

Direction of update

A binary classifier with a **sigmoid output** predicts:

$$p = 0.25, \qquad y = 1$$

Assume training uses **binary cross-entropy loss**.

Questions

1. Is the predicted probability of class 1 too high or too low?
2. Calculate the output error signal $\delta_o = p - y$.
3. Should the output score z_o increase or decrease to reduce this example's loss?
4. Explain your answer using the sign of the gradient.

FCIS _ For Course Use Only

Solution.

$$\delta_o = p - y = 0.25 - 1 = -0.75$$

The predicted probability is **too low**.

Since $\partial L / \partial z_o = -0.75$, increasing the output score **decreases the loss locally**. Gradient descent therefore moves the output score upward through suitable parameter updates.

Common Mistakes

FCIS — Personal Use Only

Neural-network errors

Architecture and preprocessing

- **No nonlinear hidden activations:** Stacked affine layers collapse into one affine transformation.
- **Ignoring input scaling:** May make optimization harder and slower.
- **Incorrect output activation or loss:** Can make predictions or training objectives inappropriate.
- **Identical hidden-weight initialization:** Neurons may learn identical representations.

Training and model selection

- **Judging only training loss:** Can hide overfitting.
- **Training too long without validation monitoring:** May worsen generalization.
- **Learning rate too large:** Can cause unstable or divergent updates.
- **Assuming neural networks always win:** Added complexity needs validation evidence.

IAIS — For Course Use Only

💡 **Final takeaway.** Good neural-network practice requires appropriate **architecture, preprocessing, optimization, regularization, and evaluation**.

Mini-Quiz

FCIS — For Course Use Only

Check your understanding

1. What are the two computations performed by a neuron?
2. Why are nonlinear activations necessary?
3. What output activation is used for binary classification?
4. What does backpropagation compute?
5. What is the output error signal for sigmoid plus binary cross-entropy?
6. Why can deep networks suffer from vanishing gradients?
7. What is early stopping?
8. Why should neural networks be compared with simpler baselines?

Only

Check your understanding

1. What are the two computations performed by a neuron?
2. Why are nonlinear activations necessary?
3. What output activation is used for binary classification?
4. What does backpropagation compute?
5. What is the output error signal for sigmoid plus binary cross-entropy?
6. Why can deep networks suffer from vanishing gradients?
7. What is early stopping?
8. Why should neural networks be compared with simpler baselines?

Suggested answers: weighted sum and activation; without nonlinearities, stacked layers remain linear; sigmoid; gradients; $p - y$; chain-rule products can become small; stop when **validation loss** stops improving; complexity must be justified by generalization and context.

Lecture 7 Summary

FCIS — For Course Use Only

Key ideas to retain

Neural-Network Mechanics

- A neuron computes $z = w^\top x + b$ and applies an activation $a = \phi(z)$.
- **Hidden layers** learn intermediate representations.
- **Nonlinear activations** allow networks to model complex relationships.
- **Output layers** depend on the task: regression or classification.
- **Forward propagation** produces predictions; the loss measures errors.
- **Backpropagation** uses the chain rule to compute gradients.

Training and Generalization

- **Gradient descent** updates weights and biases using gradients.
- **Learning rate, batch size, epochs, and initialization** influence training.
- **Training and validation curves** reveal underfitting, overfitting, and optimization difficulties.
- **Regularization, dropout, early stopping, and data augmentation** can improve generalization.
- **Validation performance and application needs** determine whether a neural network is preferable to simpler models.

FCIS — Full Course Slides

End-of-Lecture Check

FCIS — Private Use Only

Five questions to test your understanding

1. How is a **sigmoid neuron** related to logistic regression?
2. Why are **nonlinear activation functions** important in hidden layers?
3. What is the difference between **forward propagation** and **backpropagation**?
4. Why do **regression, binary classification, and multiclass classification** require different output-layer choices?
5. How can **training and validation loss curves** reveal overfitting, and what can we do about it?

FCIS — For Course Use Only

💡 **Preparation for Lecture 8.** Review **distance and similarity**, the idea of grouping observations, and **variance**. Next, we explore **clustering and dimensionality reduction**—learning useful structure without target labels.

Suggested Reading

FCIS — Private Use Only

Primary and supporting references

Primary reference

G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning with Applications in Python*, Springer, 2023.

- Review sections introducing neural networks, hidden layers, activations, training, and overfitting.

Supporting references

A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, O'Reilly, 2022.

- Review neural-network basics, training with gradient descent, activation functions, and regularization.

K. P. Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022.

- Use selected sections for deeper views of losses, gradients, probabilistic outputs, and model uncertainty.

ENS — For Course Use Only

💡 When reading, focus on the computation graph: forward pass, loss, backward gradients, and parameter updates.